

Project Report

AR Driving: An Augmented Reality Unity Project

Name: Sayna Patel

Course Name: Augmented Reality and Virtual Reality

Course Code : CSG2001

Slot : B11+B12+B13

Submitted To : Mr. Ashok Patel

GitHub Repository: <https://github.com/sayna24bey10080-gif/AR-Driving>

1. Introduction

Augmented reality (AR) blends virtual objects with the real world through a device's camera, so that digital content appears to exist in the user's physical surroundings. AR Driving is an Android application built in Unity that uses this idea to create a small driving game. The user scans a real floor or table with their phone, places a virtual car on the detected surface, and drives it around the real environment while collecting packages.

This report describes the objective of the project, the tools used, how the application works, the main challenge faced during development, and the final result. The complete source code is available on GitHub through the link above.

2. Objective

The objective of this project is to demonstrate the core building blocks of a mobile AR application:

- Plane detection: finding real-world flat surfaces such as floors and tables.
- Object placement: positioning a virtual object on a detected surface using a placement reticle.
- User interaction: letting the user control the virtual object in real time.
- Environmental awareness: adjusting virtual lighting so the object blends into the real scene.
- Version control and publishing: sharing the finished project on GitHub.

3. Tools and Technologies

Tool	Purpose
Unity ([version])	Game engine used to build the application
AR Foundation	Unity framework for cross-platform AR features
ARCore XR Plugin	Provides AR functionality on Android devices
C#	Programming language used for all scripts
Visual Studio Code	Code editor used to write and edit scripts
Git and GitHub	Version control and online project hosting
Android smartphone ([phone model])	Device used to build, install and test the app

4. System Design and How It Works

The application follows a simple flow. When it starts, the phone camera scans the surroundings and AR Foundation detects flat surfaces. A reticle appears on a detected surface to show where the car can be placed. After the user taps, the car is spawned and can be driven across the surface. Packages appear on the surface for the car to collect, and the lighting of the virtual objects is updated to match the real room.

The behavior is split across several C# scripts:

- **ReticleBehaviour:** displays a reticle on the detected surface to indicate where the car will be placed.
- **DrivingSurfaceManager:** manages the detected surface that the car drives on.
- **CarManager:** handles creating the car in the scene once the user confirms its position.
- **CarBehaviour:** controls the movement of the car.
- **PackageSpawner:** places collectible packages on the surface.
- **PackageBehaviour:** defines how a package behaves when the car interacts with it.
- **LightEstimation:** reads the lighting conditions from the camera feed and applies them to the virtual objects so the car looks more realistic.

5. Implementation Steps

1. Created a new Unity project and configured it for Android with the AR Foundation and ARCore packages.
2. Imported the starter assets and set up the AR scene with the AR session, AR camera and plane detection.
3. Configured the reticle so that it follows the detected surface.
4. Added the car and package scripts to handle spawning, driving and collecting.
5. Enabled light estimation so that virtual objects respond to real-world lighting.
6. Built the project as an APK and installed it on an Android phone for testing.
7. Initialized a Git repository, connected it to GitHub and pushed the project.

6. Challenging Task: Publishing the Project on GitHub

The most challenging task was posting the Unity project to GitHub. Unity projects contain many files and folders, and the first upload had to include the right content without the large, automatically generated folders. The project was initialized as a Git repository in VS Code's terminal, the GitHub repository was added as the remote origin, and the project was pushed with the command `git push -u origin main`.

The push required completing authentication in the browser before it would proceed. After that, Git uploaded 164 objects (about 5.29 MiB) and created the main branch on GitHub. The repository now contains the three folders Unity needs to reopen the project: Assets, Packages and ProjectSettings. As a result, anyone can clone the repository, open it in Unity Hub and build the project themselves.

7. Screenshot



8. Results

The final application runs on an ARCore-supported Android phone. It successfully detects real surfaces, places a virtual car where the user chooses, allows the car to be driven around, spawns packages to collect, and adjusts the lighting to the room. The project is published on GitHub and can be demonstrated live in the classroom.

9. Conclusion

This project showed how an AR application detects the real world, places virtual objects and responds to user input. It also gave practical experience with building Unity projects for Android and managing and publishing them with Git and GitHub. In the future, the project could be improved by adding a score counter, multiple levels, sound effects and a timer to make the game more engaging.

10. Project Links

Source code: <https://github.com/sayna24bey10080-gif/AR-Driving>

APK download: available under the Releases section of the same repository.